

# Assignment 05: Copy-on-Write Fork (cow)

CSci 530: Operating Systems

## Objectives

This lab explores the use of page table exceptions to implement copy-on-write for the `fork()` system call. Virtual memory provides a level of indirection: the kernel can intercept memory references by marking PTEs invalid or read-only, leading to page faults, and can change what addresses mean by modifying PTEs. There is a saying in computer systems that any systems problem can be solved with a level of indirection. This lab explores an example: copy-on-write fork.

Before you start coding, read Chapter 5 Page Faults of the xv6 book, of course especially the discussion of how to implement copy-on-write fork, and related source files to understand lazy memory allocation in xv6:

- `kernel/sysproc.c`: the `sys_sbrk()` function
- `kernel/vm.c` and `kernel/trap.c` for lazy memory allocation

## Description

Have a look at the Getting Started instructions for information on how to set up your computer to work on and run these assignments.

Note: Remember that assignments are submitted to GitHub classrooms by creating a commit and pushing it back to your assignment repository. You are required to make a commit for at least each task as you complete it (though you can make more than one commit for a task if needed). In general, from the command line, you can create and push a commit by doing something like:

```
$ git commit -am "My solution for Assignment 04 Task 02, backtrace"
$ git push
```

## Assignment Tasks

### Task 01: Implement copy-on-write fork (hard)

#### The problem

The `fork()` system call in xv6 copies all of the parent process's user-space memory into the child. If the parent is large, copying can take a long time. Worse, the work is often largely wasted: `fork()` is commonly followed by `exec()` in the child, which discards the copied memory, usually without using most of it. On the other hand, if both parent and child use a copied page, and one or both writes it, the copy is truly needed.

#### The solution

Your goal in implementing copy-on-write (**COW**) `fork()` is to defer allocating and copying physical memory pages until the copies are actually needed, if ever. COW `fork()` creates just a pagetable for the child, with PTEs for user memory pointing to the parent's physical pages. COW `fork()` marks all the user PTEs in both parent and child as read-only. When either process tries to write one of these COW pages, the CPU will force a page fault. The kernel page-fault handler detects this case, allocates a page of physical memory for the faulting process, copies the original page into the new page, and modifies the relevant PTE in the faulting process to refer to the new page, this time with the PTE marked writeable. When the page fault handler returns, the user process will be able to write its copy of the page.

COW `fork()` makes freeing of the physical pages that implement user memory a little trickier. A given physical page may be referred to by multiple processes' page tables, and should be freed only when the last reference disappears. In a simple kernel like xv6 this bookkeeping is reasonably straightforward, but in production kernels this can be difficult to get right; see, for example, [Patching until the COWs come home](#).

## Implementation

Your task is to implement copy-on-write fork in the xv6 kernel. You are done if your modified kernel executes both the `cowtest` and `usertests -q` programs successfully.

To help you test your implementation, we've provided an xv6 program called `cowtest` (source in `user/cowtest.c`). `cowtest` runs various tests, but even the first will fail on unmodified xv6. Thus, initially, you will see:

```
$ cowtest
simple: fork() failed
$
```

The “simple” test allocates more than half of available physical memory, and then `fork()`s. The fork fails because there is not enough free physical memory to give the child a complete copy of the parent's memory.

When you are done, your kernel should pass all the tests in both `cowtest` and `usertests -q`. That is:

```
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
$ usertests -q
...
ALL TESTS PASSED
$
```

Here's a reasonable plan of attack:

1. Modify `uvmcopy()` to map the parent's physical pages into the child, instead of allocating new pages. Clear `PTE_W` in the PTEs of both child and parent for pages that have `PTE_W` set.
2. Modify `vmfault()` to recognize page faults. When a write page-fault occurs on a COW page that was originally writeable, allocate a new page with `kalloc()`, copy the old page to the new page, and install the new page in the PTE with `PTE_W` set. Pages that were originally read-only (not mapped `PTE_W`, like pages in the text segment) should remain read-only and shared between parent and child; a process that tries to write such a page should be killed.
3. Ensure that each physical page is freed when the last PTE reference to it goes away, but not before. A good way to do this is to keep, for each physical page, a “reference count” of the number of user page tables that refer to that page. Set a page's reference count to one when `kalloc()` allocates it. Increment a page's reference count when `fork()` causes a child to share the page, and decrement a page's count each time any process drops the page from its page table. `kfree()` should only place a page back on the free list if its reference count is zero. It's OK to keep these counts in a fixed-size array of integers. You'll have to work out a scheme for how to index the array and how to choose its size. For example, you could index the array with the page's physical address divided by 4096, and give the array a number of elements equal to highest physical address of any page placed on the free list by `kinit()` in `kalloc.c`. Feel free to modify `kalloc.c` (e.g., `kalloc()` and `kfree()`) to maintain the reference counts.
4. Modify `copyout()` to use the same scheme as page faults when it encounters a COW page.

Some hints:

- It may be useful to have a way to record, for each PTE, whether it is a COW mapping. You can use the RSW (reserved for software) bits in the RISC-V PTE for this.

- `usertests -q` explores scenarios that `cowtest` does not test, so don't forget to check that all tests pass for both.
- Some helpful macros and definitions for page table flags are at the end of `kernel/riscv.h`.
- If a COW page fault occurs and there's no free memory, the process should be killed.

## Submit the Assignment

### Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. Make sure you add and commit a final commit with this file as it is part of the autograder score.

Assignment submissions are handled by the GitHub classroom autograder. You should create at least 1 commit for each task as you do it, and push it to your GitHub classroom. You can check that the autograder tests are passing in GitHub classroom by looking at your submitted releases. The tests run in the GitHub classroom are the same ones that you can run locally by using `make grade` or `./grade-lab-cow test`.

- Please run `make grade` to ensure that your code passes all of the tests. The GitHub classroom autograder will use the same grading program to assign you an autograder evaluation score for your work.

## Optional Challenge Exercises

- Measure how much your COW implementation reduces the number of bytes xv6 copies and the number of physical pages it allocates. Find and exploit opportunities to further reduce those numbers.